

Documentation, Type Hints, and Composing Functions

Day 4 Morning - Functions and Program Design

30 min teaching + 30 min exercises

```
def calculate_percentage(  
    completed: int,  
    total: int  
) -> float:  
    """Calculate completion  
percentage."""  
    return completed / total * 100
```

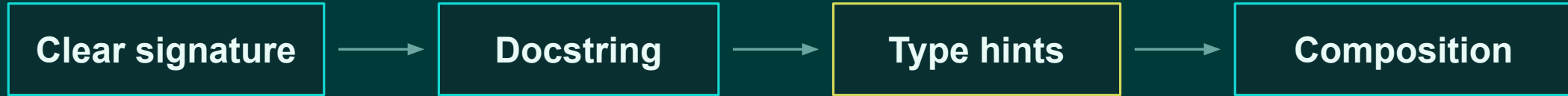
```
normalize_record()  
    ↓  
validate_record()  
    ↓  
process_record()
```

Goal: make function behavior easier to read, reuse, and combine into workflows.

BLOCK 28

Where this block fits

We now make functions self-explanatory and compose them into processing stages.



Instruction focus

- function contracts
- docstrings
- type hints
- reading signatures
- calling functions from functions
- main-program structure

Practice focus

Students add documentation and type hints, then build one larger function from several smaller ones.

BLOCK 28

A function has a contract

A contract explains what the function expects and what it gives back.

```
def calculate_percentage(completed, total):
```

Mental model

A function contract is the “agreement” between the caller and the function.

Contract

Input:

completed: completed records
total: total records

Requirement:

total must be greater than zero

Output:

completion percentage as a float

Before writing a function, ask: what promise is this function making?

BLOCK 28

Four questions for every function

These questions prevent vague, surprising function behavior.

1. Inputs

What does this function need?

2. Output

What value does this function return?

3. Invalid values

What values are not meaningful?

4. Side effects

Does it change anything outside itself?

Design habit

If you cannot answer these quickly, the function may be too vague or too large.

BLOCK 28

Docstrings explain purpose and assumptions

A docstring is the short explanation inside the function.

```
def calculate_percentage(completed, total):  
    """Calculate the percentage of completed  
items."""  
    return completed / total * 100
```

```
def calculate_percentage(completed,  
total):  
    """  
        Calculate completion percentage.  
  
        The total value must be greater than  
zero.  
    """  
    return completed / total * 100
```

Beginner rule

Important functions should explain their purpose and assumptions.

BLOCK 28

Docstrings are not comments for obvious code

Use them to explain the function's role, not every small step.

Less useful

```
"""Multiply completed by total."""
```

This restates code and may even be wrong.

More useful

```
"""Return completion percentage.
```

```
The total must be greater than zero."""
```

Good docstrings answer
Why does this function exist?
What assumptions matter?
What does the caller get back?

BLOCK 28

Type hints communicate expected types

They make the function signature more informative.

```
def calculate_percentage(  
    completed: int,  
    total: int  
) -> float:  
  
    return completed / total * 100
```

completed: int
completed should be an integer.

total: int
total should be an integer.

-> float
the function is expected to return a float.

A signature should tell the reader how to call the function correctly.

BLOCK 28

Type hints do not enforce runtime safety

Python still runs dynamically unless extra tools check the hints.

```
def double_value(number: int) -> int:  
    return number * 2  
  
print(double_value("ha"))
```

What type hints do
Communicate intent to developers, editors, documentation systems, and static analysis tools.

What they do not do
Automatically stop every wrong value at runtime.

Practical takeaway
Use type hints as a readability tool first. Later, tools can verify them.

BLOCK 28

Boolean-returning functions should say so

The return hint helps readers know this function answers yes/no.

```
def is_supported_file(file_name: str) -> bool:
    """Return True when the file type is
    supported."""

    normalized_name = file_name.strip().lower()

    return (
        normalized_name.endswith(".csv")
        or normalized_name.endswith(".json")
    )
```

Read the signature
Input: file_name is a string
Output: True or False

Naming pattern
is_...() usually returns a
Boolean.

Good names and type hints reinforce each other.

BLOCK 28

Reading a function signature

A signature is a compact summary of how to use a function.

```
def classify_error_count(error_count: int) -> str:
```

Function name

`classify_error_count`

The function classifies an error count.

Parameter

`error_count: int`

The caller supplies an integer.

Return type

`-> str`

The function returns text.

Instructor prompt

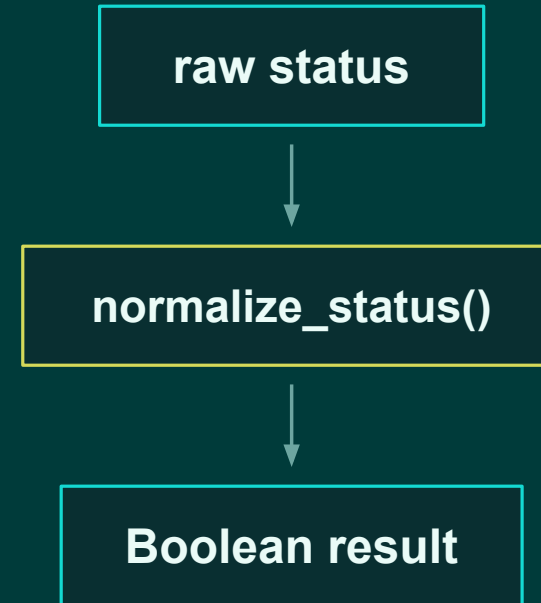
Ask students to predict possible return values before showing the function body.

BLOCK 28

One function can call another

Small functions can be combined into more useful operations.

```
def normalize_status(status: str) -> str:  
    return status.strip().lower()  
  
def is_active(status: str) -> bool:  
    normalized_status = normalize_status(status)  
  
    return normalized_status == "active"
```



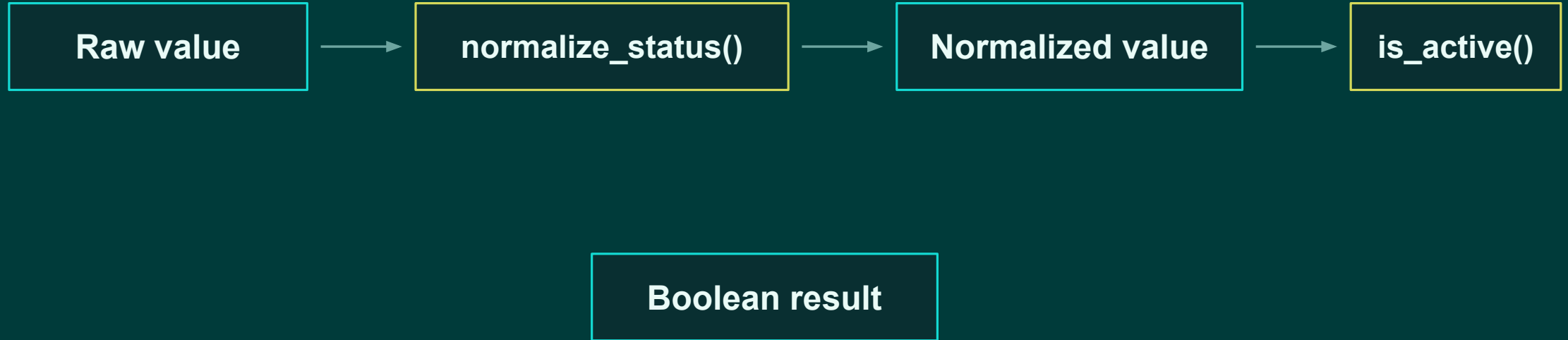
Composition

Build a more specific function from a smaller general-purpose function.

BLOCK 28

Composition creates layers of abstraction

Each layer hides one small detail and gives the next layer something cleaner.



Why this helps

The main code no longer needs to remember stripping whitespace, lowercasing, and comparing all at once.

BLOCK 28

Record processing as a pipeline

A multi-record program can be made of clear stages.



Pipeline habit

A complex program becomes easier to read when each stage has a clear input and output.

BLOCK 28

Separate capabilities from main execution

Define functions first; use them in the main program after.

```
def normalize_status(status: str) -> str:
    return status.strip().lower()

def is_valid_status(status: str) -> bool:
    allowed_statuses = {
        "active",
        "inactive",
        "retired",
    }

    return status in allowed_statuses
```

Top section
Defines capabilities.

```
status = " ACTIVE "

clean_status = normalize_status(status)
valid_status =
is_valid_status(clean_status)

print(clean_status)
print(valid_status)
```

Bottom section
Uses those capabilities.

BLOCK 28

A cleaner main program reads like a summary

The low-level details live inside well-named functions.

```
processed_records = []  
  
for record in records:  
    processed_record = process_record(record)  
  
processed_records.append(processed_record)
```

What the reader sees
For each record:
 process it
 save the processed result

Design goal

The main program should describe what is happening without exposing every implementation detail at once.

BLOCK 28

Guided exercise setup

Add documentation and type hints, then compose functions into `process_record()`.

Starting functions

```
normalize_record_id()
normalize_status()
calculate_percentage()
is_supported_file()
classify_error_count()
```

Time plan

```
Exercise 1: 8 min
Exercise 2: 7 min
Exercise 3: 10 min
Applied challenge: remaining time
```

Practice rule

Do not make functions bigger just to avoid calling another function.

BLOCK 28

Exercise 1: add docstrings and type hints

Turn earlier functions into documented function contracts.

```
def normalize_record_id(record_id):  
    return record_id.strip().upper()  
  
def calculate_percentage(completed, total):  
    return completed / total * 100
```

Task

Add parameter type hints, return type hints, and one useful docstring to each function.

8 minutes

Checkpoint

Can another student tell what each function expects without reading the whole body?

BLOCK 28

Exercise 2: document a Boolean helper

Make the signature and docstring clearly say yes/no behavior.

```
def is_supported_file(file_name):  
    file_name = file_name.strip().lower()  
  
    return (  
        file_name.endswith(".csv")  
        or file_name.endswith(".json")  
    )
```

Task

Add:

- file_name: str
- -> bool
- a docstring that explains supported file types

7 minutes

Stretch

Write three quick calls that should return True, True, and False.

BLOCK 28

Exercise 3: compose process_record()

Create a function that calls smaller functions.

```
def process_record(record: dict) -> dict:

    record_id = normalize_record_id(
        record["record_id"]
    )

    status = normalize_status(
        record["status"]
    )

    processing_status = classify_error_count(
        record["error_count"]
    )

    return {
        "record_id": record_id,
        "status": status,
        "processing_status": processing_status,
    }
```

Task

Build this function from smaller functions you already wrote.

10 minutes

Check

Call process_record() with one sample dictionary and inspect the returned dictionary.

BLOCK 28

Applied challenge: refactor the Day 3 processor

Reorganize the multi-record workflow using documented functions.

Minimum functions

`normalize_record()`

`validate_record()`

`classify_record()`

`process_record()`

```
processed_records = []
```

```
for record in records:
```

```
    processed_record =  
    process_record(record)
```

```
processed_records.append(processed_record)
```

Task

Choose one large Day 3 loop. Move one responsibility at a time into a named function.

Remaining time

BLOCK 28

Challenge success criteria

Use this checklist before calling the refactor complete.

Each function has

- a clear name
- explicit parameters
- a return value
- useful type hints
- a short docstring

The main loop has

- no hidden normalization details
- no giant validation block
- no repeated classifications
- readable `process_record()` calls

Final check

Can you test `normalize_record()`, `validate_record()`, and `classify_record()` without running the entire program?

BLOCK 28

Block 28 wrap-up

Students should now see functions as documented, typed, composable building blocks.

Students can explain

- function contracts
- inputs and outputs
- preconditions
- docstrings
- type hints
- function signatures

Students can do

- call one function from another
- build a processing pipeline
- separate definitions from main code
- compose `process_record()`
- refactor a Day 3 loop

Next: debugging, tracebacks, exception handling, and testing.